

بِسْمِ اللَّهِ الرَّحْمَنِ الرَّحِيمِ

# COMPUTER GRAPHICS

**Presented by: Eng.Karama**

On Facebook, Twitter, Instagram:

@acmelacademy or  
email:acmelacademy@gmail.com

# SCOPE

## FILLED AREA PRIMITIVES

---

Seed fill Overview

---

Flood fill algorithm

---

4 connected and 8 connected pixels approaches

---

Flood fill algorithm steps



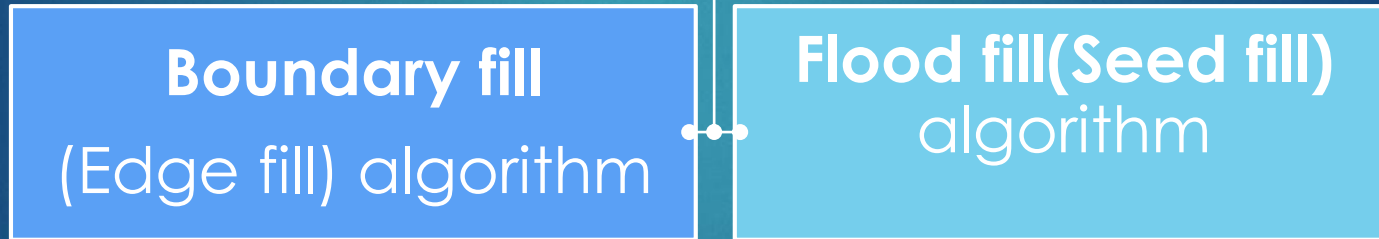
# SEED FILL ALGORITHMS

## FLOOD FILL

© 2013 - 2026 akmel online

# SEED FILLING OVERVIEW?

Is one of polygon filling approaches by starting from a given “seed” inside the polygon and highlight outward from this point to boundary pixels.



# FLOOD FILLING ALGORITHM?

This algorithm picks a point inside an object and start to fill until it hits the boundary of the object similar to the boundary fill but it can be used when the boundary color is not single-color.



## BASICS

Flood fill algorithm is useful in cases where there no single color boundary for the polygon, i.e. the boundary has multiple colors.

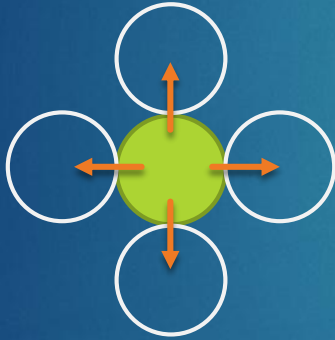
In flood fill algorithm instead of filling color till you encounter a specific boundary color you just fill the pixels with default color.

It is used in the "bucket" fill tool of paint programs to fill connected, similarly-colored areas with a different color.



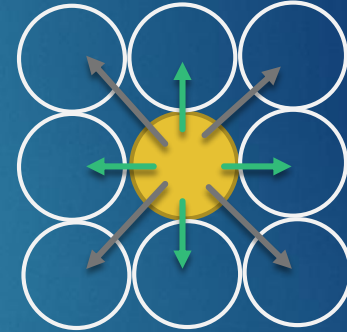
# 4 CONNECTED PIXELS OR 8 CONNECTED PIXELS APPROACHES

4 connected pixels or 8 connected pixels approaches are used to fill the boundary.



## **4 connected region**

Every pixel can be reached  
by a combination of moves  
in FOUR directions:  
(Left, Right, Up, Down)



## **8 connected region**

Every pixel can be reached  
by a combination of moves  
in EIGHT directions:  
(2 horizontals, 2 verticals  
and 4 diagonals)





# FLOOD FILL ALGORITHM STEPS

First initialize the 4 values:

x, y, Fill-color & Default-color.

If the color of node is not **equal** to the **default** color, Return

Check if the current pixel is **Default-color** and if **Yes** then Repeat **Step 4** and **step 5** till the boundary pixels are reached.

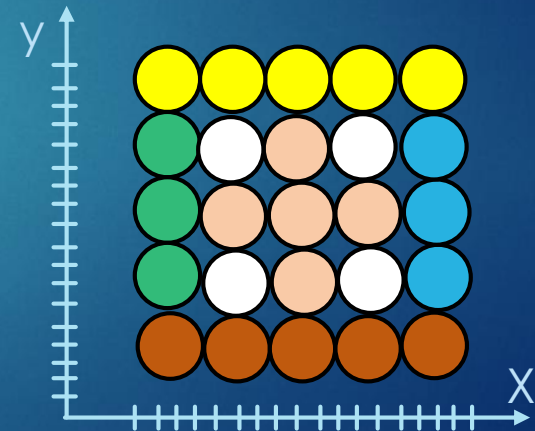
Repeat step 3 and step 4

for the neighboring 4 pixels.

Change the default color with fill color at the current pixel

```
void FloodFill4(x,y,fill, default)
{ int current = getpixel(x,y)
  if (current == default)
  {
    setpixel(x,y,fill);
    setpixel(x,y,fill);
    FloodFill4(x+1,y,fill,default);
    FloodFill4(x-1,y,fill, default);
    FloodFill4(x,y+1,fill, default);
    FloodFill4(x,y-1,fill, default);
  }
}
```

4 Connected Fill Approach





# FLOOD FILL ALGORITHM STEPS

First initialize the 4 values:

x, y, Fill-color & Default-color.

If the color of node is not **equal** to the **default** color, Return

Check if the current pixel is of **Default-color** and if **Yes** then Repeat **Step 4** and **step 5** till the boundary pixels are reached.

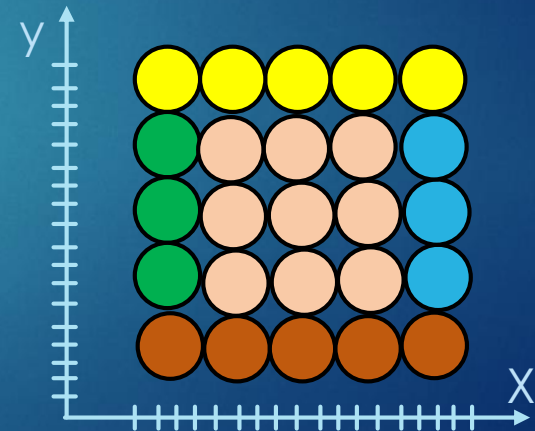
Repeat step 3 and step 4

for the neighboring 4 pixels.

Change the default color with fill color at the current pixel

```
void FloodFill8(x,y,fill, default)
{ int current = getpixel(x,y)
  if (current == default)
  {
    setpixel(x,y,fill);
    setpixel(x,y,fill);
    FloodFill4(x+1,y,fill,default);
    FloodFill4(x-1,y,fill, default);
    FloodFill4(x,y+1,fill, default);
    FloodFill4(x,y-1,fill, default);
    FloodFill4(x+1,y+1,fill,default);
    FloodFill4(x+1,y-1,fill, default);
    FloodFill4(x-1,y+1,fill, default);
    FloodFill4(x-1,y-1,fill, default);
  }
}
```

8 Connected Fill Approach





**MORE PRACTICE MAKES  
YOU PERFECT**





# Always Seek Knowledge



# THE END

